2025 International Conference on Emerging Trends in Industry 4.0 Technologies (ICETI4T) | 979-8-3315-0696-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICETI4T63625.2025.11132233

Workflow Engines for Task Automation: Taxonomy and Analysis

Nikita Pawar
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
nikita.pawar@cumminscollege.in

Shilpa Deshpande
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
shilpa.deshpande@cumminscollege.in

Aparna Hajare
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
aparna.hajare@cumminscollege.in

Sakshi Wagh
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
sakshi.wagh@cumminscollege.in

Sanskruti Wakode
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
sanskruti.wakode@cumminscollege.in

Aastha Thorat
Department of Computer Engineering
Cummins College of Engineering for Women
Pune, India
aastha.thorat@cumminscollege.in

Vijay Dulange
Veritas Software Technologies India Private Limited
Pune, India
vijaykumar.dulange@veritas.com

***Abstract*—Workflow engines are tools designed to automate and coordinate tasks across different systems, making it easier to manage complex processes. They simplify processes, reducing manual effort and helping organizations focus on their core objectives. There are varying demands of complex applications in the enterprise era, which need to be fulfilled by workflow engines. This paper presents the taxonomy of workflow engines and proposes a structured approach to analyze their strengths and weaknesses, enabling organizations to identify the most effective solution for their automation requirements. The intent of this paper is also to identify core capabilities required for the effective automation of tasks using workflow engines.**

Keywords— Deployment Model, Execution Paradigm, Programming Model, Task Automation, Use Case, Workflow Engine.

I. Introduction

In Today's world, large scale enterprise projects in various domains involve complex business operations and data processing tasks. Manually carrying out different tasks in such projects becomes effort-intensive and time consuming. Consequently, automating the execution of various tasks and simplifying the complex processes in the projects are highly essential. In this context, workflow engines are the significant mechanisms [1] which facilitate stepwise automation in projects and thereby increasing the efficiency and reliability.

The automation of growing scale of business operations and data management enabled by workflow engines minimizes human errors and thus improves consistency. Various scenarios such as orchestrating the stages in machine learning, managing data transformation and approval processes are the examples where workflow engines can be applied effectively [2].

The objective of this paper is to provide a comprehensive taxonomy of workflow engines, discussing their types, advantages, challenges, and key characteristics. By doing so, it aims to help organizations make informed decisions on the most effective systems for their automation requirements [2].

This paper proposes a structured approach to analyze the strengths and weaknesses of different workflow engines, offering insights into how they can be optimized to meet evolving organizational needs and objectives.

The paper is structured as follows. In Section II, the classification criteria of workflow engines are discussed. Section III highlights the core capabilities of workflow engines. Section IV provides an analysis of various workflow engines, examining their strengths, weaknesses, and unique features. Section V concludes the paper.

II. Classification Criteria For Workflow Engines

Workflow engines are essential tools in automating and managing tasks across various domains, including security, backup, distributed systems, cloud environments, and enterprise applications. They help streamline processes and improve efficiency by coordinating and executing workflows in a structured manner. Baïna [2] classifies workflow engines based on various categories. We build upon this classification and extend it further to address the

979-8-3315-0696-4/25/$31.00 ©2025 IEEE

diverse and evolving needs of organizations. We classify workflow engines based on architecture, deployment, programming model, execution paradigm, primary use cases, and community support. As a result, a detailed taxonomy of workflow engines is developed. This taxonomy is shown in Fig. 1.

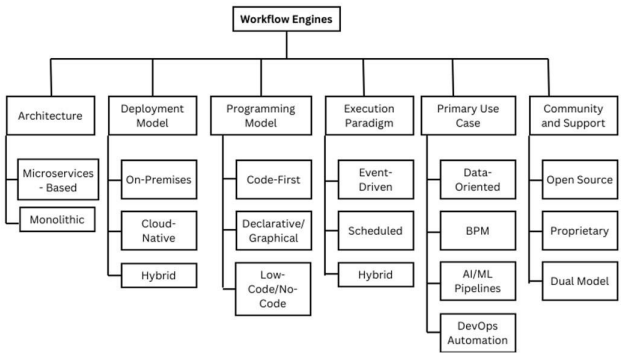

Fig. 1. Taxonomy of Workflow Engines

Fig. 1 illustrates the taxonomy of workflow engines, highlighting the various categories and classifications. The details are described as below.

A. Architecture

The architecture of workflow engines refers to the underlying structure that defines how workflows are designed, executed, and managed. Different organizations have different needs and hence by the architectural overview in the aspects like scalability, performance and flexibility, we can choose the best suitable option. The workflow architecture has components like core workflow engine and user interface. It also supports integration with external systems. Monolithic and microservices are two different types of architecture, each having various advantages.

1) Microservices-Based

Deploying and scaling components independently nowadays plays a major role to improve performance of the large-scale systems in the aspects of flexibility and fault tolerance. Microservices-Based architectures are used for the same using distributed environment. Temporal [3], Cadence [4], and Argo Workflows [5] are examples of such workflow engines.

2) Monolithic

In monolithic architecture all the components act as a unified system. This architecture makes deployment and management easier but faces challenges for scalability and flexibility. IBM BPM [6] and Tibco BPM [7] are examples of such workflow engines.

B. Deployment Model

Crucial features like control over data, security and infrastructure management are impacted by this model. Deployment speaks about the way and the time when the engine is hosted and executed.

With varied requirements like regulatory compliance, dynamic scaling and flexibility, organization chooses the most appropriate model. The three different categories of deployment model are listed below.

1) On-Premises

Control over data security and compliance is achieved through the on-premises deployment as it installs and executes the workflow engines within private infrastructure of an organization, hence this model is useful for industries with strict regulatory compliance. Activiti [8] and Bonita BPM [9] are examples of such workflow engines.

2) Cloud-Native

Nowadays workloads are very dynamic and hence they require cloud features like scalability, serverless execution and high availability [10]. Cloud native model is perfect for such situations. Amazon Step Functions and Azure Logic Apps are examples of such workflow engines.

3) Hybrid

Organization sometimes requires processes within private infrastructure and also cloud features for workload management. Hybrid models support both and offer flexibility of transitioning between environments or a mixed infrastructure. Camunda [11] and KubeFlow Pipelines [12] are examples of such workflow engines.

C. Programming Model

The definition and implementation of workflow affect level of control, technical expertise and customization which are necessary things for creating and managing the workflows which are determined by the programming model. Three different programming models are listed below.

1) Code-First

The workflows which are designed programmatically give granular control and customization making it most appropriate for complex automation that needs integration with existing systems. This type of workflow comes under code-first approach. Temporal [3] and Flyte [13] are examples of such workflow engines.

2) Declarative/Graphical

Users having business knowledge or non-developers who do not have in-depth technical expertise require workflow creation through simple ways. This engine gives visual interfaces and declarative methods to create such workflows. Camunda [11] and Bonita BPM [9] are examples of such workflow engines.

3) Low-Code/No-Code

For non-technical users the easiest way for creation of workflows is drag and drop method which is given by low-code or no-code programming model. This method is most suitable for quick and easy execution of workflows. Zapier and Microsoft Power Automate are examples of such workflow engines.

D. Execution Paradigm

The execution and triggering of workflows is given by execution paradigm. The measures for the evaluation of adaptation of different types of tasks include responsiveness, timing, and flexibility of the workflows, which are affected

by the execution paradigm. Three different types of execution paradigm are listed below.

1) Event-Driven

Depending on the internal or external events triggering the workflow comes under event-driven execution paradigm. The system that requires real time trigger and response mostly uses this paradigm as it is most appropriate for the same. Temporal [3] and StackStorm [14] are examples of such workflow engines.

2) Scheduled

The execution of workflow takes place based on predefined times or intervals. This type of execution is appropriate for batch jobs or periodic data processing tasks. Apache Airflow [15] and Luigi [16] are examples of such workflow engines.

3) Hybrid

Hybrid engines are combination of both event-driven and scheduled executions which gives edge to diverse use cases and flexibility. StackStorm [14] and Argo Workflows [5] are examples of such workflow engines.

E. Primary Use Case

The domain or a specific task in some application is referred as the primary use case of workflow engines. The streamlining of business and technical operations determines the type of workflow required which can be categorized in different types of use cases for workflow engines which are as follows.

1) Data-Oriented Workflows

The processes required for data pipelines such as Extract, Transform and Load (ETL) and analytics [17] come under the application of data-oriented workflows. Making sure of scalability and reliability in data-intensive environments, these workflows handle complex data processing very effectively. Apache Airflow [15, 18] and Kubeflow Pipelines [12] are examples of such workflow engines.

2) Business Process Management (BPM)

Business goals are achieved with the help of tools like modeling, monitoring and managing workflows. This tool helps automating business processes and achieve goals, and hence this use case comes under Business Process Management. Camunda [11] and Bonita BPM [9] are examples of workflow engines used for the BPM use case.

3) Artificial Intelligence/Machine Learning (AI/ML) Pipelines

Machine learning models include various tasks such as model training, deployment and monitoring, these engines help automate and optimize (AI/ML) workflows. KubeFlow Pipelines [12] and Flyte [13] support (AI/ML) use case.

4) DevOps Automation

Continuous Integration and Continuous Deployment (CI/CD) pipelines, streamlining development and operations processes are tasks in DevOps which are managed by these workflow engines. Argo Workflows [5] and StackStorm [14] support DevOps automation use case.

F. Community and Support

The community and support model of workflow engines refers to the availability for updates, documentation, and troubleshooting resources which ensures maintenance and user assistance. Community-driven or commercial are categories determined by this model giving various level of customization and support for users. The community and support models for workflow engines are as follows.

1) Open Source

These engines are community-driven which give active community support and are also open-source which is a cost-effective way to get solutions. This helps in continuous development and innovation. Apache Airflow [15] and n8n [19] are examples of open-source model.

2) Proprietary

Features like dedicated support, and service-level agreements, often at a higher cost are required at enterprise level. These features that require advanced capabilities with vendor-backed expertise are offered by proprietary workflow engines. IBM BPM [6] and Tibco BPM [7] are examples of such workflow engines.

3) Dual Model

These engines support both open-source as well as proprietary model giving a large scale for applications, from enterprise to open-source level based on specific needs. Camunda [11] is an example of dual model.

III. Core Capabilities Of Workflow Engines

Workflow engines play a crucial role in automating and managing complex processes across diverse projects, helping to enhance efficiency and streamline operations. As the increasing needs of projects demand greater flexibility and scalability, a versatile workflow engine must be designed with core capabilities to address the diverse and evolving requirements of enterprise environments, including stringent security measures, efficient workload management, and adaptability to emerging demands. It should seamlessly handle varying project requirements by integrating advanced features like dynamic task orchestration, scalability, error handling, and workload optimization. This adaptability ensures the workflow engine remains relevant and effective in meeting the complex challenges of modern enterprise environments. We identify and elaborate on these various capabilities as follows.

A. Security

Security ensures the protection of the sensitive data and access to authorized users. Data security and access control are very important in workflow engines. Hence features like Role-Based Access Control (RBAC) restrict access based on roles, thus preventing misuse which ensures the security. While transmission and storage of the data, encryption makes sure the data remains secure. Without these actions the data is at risk and also workflow engine will have to face legal consequences.

B. Integration Capabilities

Integration is the main part of execution. Integration with IT systems, databases, APIs, and third-party services is crucial. This flexibility enables workflows to be easily adapted and scaled according to the requirements, optimizing the actions without doing much change in the system. Workflows would be less effective and effortful to adapt without integration.

C. Versioning

For a user, consistency and reliability matter a lot, hence handling changes is a very important part. Versions of workflow need to be recorded so as to help teams track modifications, roll back to earlier versions if required and smooth functioning throughout the workflow lifecycle. Errors can be minimized using version control and stability can be achieved across different stages without losing control.

D. Error Handling

Error handling is a necessary part in every system. Workflow to remain resilient and for fast recovery from failures, error handling is required which prevents disruptions and maintains consistency. Smooth operation of workflow is ensured by error handling. Automatic retries for failed tasks and propagation of errors to dependent tasks are the major activities error handling does. Manual intervention takes place wherever necessary.

E. Scalability

As demand and workloads increase, scalability comes into picture. Two types of scaling are possible. Horizontally which means adding more nodes and vertically which improves the existing resources. By doing so the system makes sure of performance and efficiency. Scalability helps maintain smooth operations even in high demands.

F. Task Parallelism

The system is considered efficient when it takes less time for execution. Task parallelism supports execution of multiple tasks simultaneously which reduces overall time and improves efficiency [20]. The workflows with heavy data and computationally intensive tasks are executed faster with better resource utilization.

G. Task Scheduling

Batch processing, periodic data tasks and time-sensitive operations require automated process to trigger the workflow at specific times or intervals using task queues to manage the order for execution [21]. This makes sure the tasks are completed on time which helps to meet deadline and improves efficiency. Delays in the workflow or inefficiencies may occur without proper scheduling which will affect overall performance and reliability.

IV. Analysis Of Workflow Engines

Various domains require automation for tasks to get completed in effective and efficient manner ensuring the reliability as well. Diverse workloads including data pipelines and machine learning workflows can execute the tasks efficiently because of features like task orchestration, error handling, and scalability. By minimizing the manual intervention and streamlining operations, the productivity would enhance and meet specific organizational needs. Some of the most efficient workflow engines are described below in this section.

Temporal specializes in handling long-running, resilient processes, making it highly reliable but tailored to niche use cases with a focus on workflow states [3]. Argo Workflows optimizes for parallel execution in Kubernetes environments [22], providing cloud-native efficiency but demanding Kubernetes expertise [5]. Kubeflow Pipelines excels in cloud-based machine learning tasks with its Kubernetes foundation and containerized design but is less suited for non-ML workflows and requires expertise [12]. Flyte focuses on reproducibility and result sharing in ML and data workflows, making it well-suited for experimental setups, though it remains highly specialized [13].

Apache Airflow stands out for managing complex workflows with its use of Directed Acyclic Graphs (DAGs) and strong community support, though it can be resource-intensive and lacks robust event-driven capabilities [15, 23]. n8n offers a low-code, visual solution for workflow automation, making it user-friendly for non-technical users, though it lacks the scalability and advanced features of other engines [19]. Prefect's Python-first approach offers dynamic task adjustments and strong failure recovery, making it ideal for Python developers, though it lacks the robustness of more mature platforms [24].

Dagster emphasizes data accuracy with strong typing and modular design, making it flexible for data-centric tasks while being overly strict for simpler workflows [25]. Zeebe's event-driven nature and Business Process Model and Notation (BPMN) design cater to microservices, excelling in asynchronous tasks but limiting its broader applicability [26]. Apache DolphinScheduler provides a drag-and-drop interface for non-technical users, supporting custom plugins but struggling with scalability [27].

Table I provides a comparison of workflow engines, detailing their methodologies, strengths, and limitations to help identify the best fit for various needs. It highlights key capabilities like scalability and error handling, showcases strengths such as integration and fault tolerance, and addresses drawbacks like resource demands or limited scope, offering a balanced view for informed decision- making. As part of the critical analysis, the limitations of each workflow engine are outlined in the table, emphasizing the need for further exploration to address these gaps and enhance the applicability of the engines discussed in Table I.

V. Conclusion

This paper presents a taxonomy of workflow engines, focusing on their architecture, programming models, deployment methods, and use cases. Further this work also identifies the core capabilities of workflow engines which are essential for considering the diverse need of enterprise applications. Moreover in this paper we also analyze existing workflow engines based on their relative strengths and limitations. This analysis implies that workflow engines cater to diverse needs, ranging from data processing and

business workflows to machine learning and CI/CD pipelines. At the same time, it also shows limitations such as scalability issues in handling large datasets, difficulty in integrating with legacy systems, and challenges in ensuring fault tolerance in complex workflows, which need to be addressed. As distributed systems grow in complexity, workflow engines must evolve to support multi-cloud environments, real-time monitoring, and AI-driven optimization, opening avenues for further research and innovation.

TABLE I. ANALYSIS OF WORKFLOW ENGINES

Workflow Engine	Methodology	Strengths	Limitations
Temporal [3]	- Long-running tasks are handled using retries.	- Great for long-running processes. - Very reliable. - High throughput and resilience.	- Requires understanding of workflow states. - Niche use cases.
Argo Workflows [5]	- Executes workflows as Kubernetes pods. - Deep Kubernetes integration.	- Cloud-native efficiency. - Optimized for parallel execution.	- Not ideal outside Kubernetes. - Requires Kubernetes expertise.
Kubeflow Pipelines [12]	- Built on Kubernetes. - Uses containerized tasks.	- Focused on ML tasks. - Supports auto-scaling. - Scales well with Kubernetes.	- Not suited for non-ML workflows. - Steep learning curve for beginners.
Flyte [13]	- Orchestrates workflows with a focus on reproducibility and data sharing.	- Ideal for ML experiments. - Strong reproducibility.	- Highly specialized. - Smaller community.
Apache Airflow [15]	- Uses DAGs for task dependencies. - Automatic retries and failure notifications.	- Great for complex workflows. - Strong community support. - Supports dynamic scheduling and parallel tasks.	- Limited event-driven support. - Can be resource-heavy.
n8n [19]	- Open-source, low-code workflow automation.	- Easy-to-use visual interface. - Great for non-technical users. - Supports integration with various services.	- Limited scalability for large workflows. - May lack advanced features compared to other engines.
Prefect [24]	- Python-first design. - Dynamic task adjustments.	- Easy for Python developers. - Versatile for different workflows. - Strong failure recovery.	- Less robust than mature platforms. - Smaller ecosystem.
Dagster [25]	- Modular design for reusability. - Strong typing for task validation.	- Ensures data accuracy. - Flexible for data tasks.	- Strict for simple workflows. - Smaller user base.
Zeebe [26]	- Event-driven workflows for microservices. - Uses BPMN for design.	- Perfect for microservices. - Handles asynchronous tasks well.	- Less effective for outside microservices. - Needs BPMN knowledge.
Apache DolphinScheduler [27]	- Drag-and-drop interface. - Custom plugins support.	- Easy for non-technical users. - Flexible with task extensions.	- Limited scalability. - Smaller ecosystem.

References

[1] S. Zainudin and A. R. Hamdan, "A proposed design for a workflow engine," Proc. IEEE Region 10 Int. Conf. Electr. Electron. Technol. (TENCON), vol. 1, pp. 65–68, 2001

[2] Baına, Karim. "A tool for comparing and selecting workflow engines." (2007).

[3] https://temporal.io/docs/ [Accessed on: 1st December, 2024]

[4] https://cadenceworkflow.io/ [Accessed on: 2nd December, 2024]

[5] https://argoproj.github.io/argo-workflows/ [Accessed on:3rd December, 2024]

[6] https://www.ibm.com/products/business-process-manager [Accessed on: 4th December, 2024]

[7] https://www.tibco.com/products/tibco-bpm-enterprise [Accessed on: 5th December, 2024]

[8] https://www.activiti.org/ [Accessed on: 6th December, 2024]

[9] https://www.bonitasoft.com/ [Accessed on: 8th December, 2024]

[10] Stéphane, Fouakeu-Tatieze, Kamla Vivient Corneille, Sonia Yassa, Kamgang Jean-Claude, and Nkenlifack Marcellin Julius Antonio. "Taxonomy and survey of scientific workflow scheduling in infrastructure-as-a-service cloud computing systems." *Journal of Advanced Computer Science & Technology* 12, no. 1 (2024): 19-32.

[11] https://camunda.com/ [Accessed on: 10th December, 2024]

[12] https://www.kubeflow.org/docs/components/pipelines/ [Accessed on: 11th December, 2024]

[13] https://flyte.org/ [Accessed on: 12th December, 2024]

[14] https://stackstorm.com/ [Accessed on: 13th December, 2024]

[15] https://airflow.apache.org/ [Accessed on: 14th December, 2024]

[16] https://luigi.readthedocs.io/ [Accessed on:15th December, 2024]

[17] Mitchell, Ryan, Loïc Pottier, Steve Jacobs, Rafael Ferreira da Silva, Mats Rynge, Karan Vahi, and Ewa Deelman. "Exploration of workflow management systems emerging features from users perspectives." In 2019 IEEE International Conference on Big Data (Big Data), pp. 4537-4544. IEEE, 2019.

[18] T. K. Turner and A. Kapoor, "Efficient ETL processes: A comparative study of Apache Airflow vs. traditional methods," Journal of ETL Processes, vol. 2, no. 4, pp. 78–88, 2022.

[19] https://n8n.io/ [Accessed on: 16th December, 2024]

[20] Erbel, Johannes, and Jens Grabowski. "Scientific workflow execution in the cloud using a dynamic runtime model." Software and Systems Modeling 23, no. 1 (2024): 163-193.

[21] Uhrin, Martin, Sebastiaan P. Huber, Jusong Yu, Nicola Marzari, and Giovanni Pizzi. "Workflows in AiiDA: Engineering a high- throughput, event-based engine for robust and modular computational workflows." Computational Materials Science 187 (2021): 110086.

[22] Shan, Chenggang, Chuge Wu, Yuanqing Xia, Zehua Guo, Danyang Liu, and Jinhui Zhang. "Adaptive resource allocation for workflow containerization on Kubernetes." Journal of Systems Engineering and Electronics 34, no. 3 (2023): 723-743.

[23] Shi, Juwei, and Jiaheng Lu. "Performance models of data parallel DAG workflows for large scale data analytics." Distributed and Parallel Databases 41, no. 3 (2023): 299-329.

[24] https://www.prefect.io/ [Accessed on: 17th December, 2024]

[25] https://www.dagster.io/ [Accessed on: 18th December, 2024]

[26] https://zeebe.io/ [Accessed on: 20th December, 2024]

[27] https://dolphinscheduler.apache.org/ [Accessed on: 21st December, 2024]